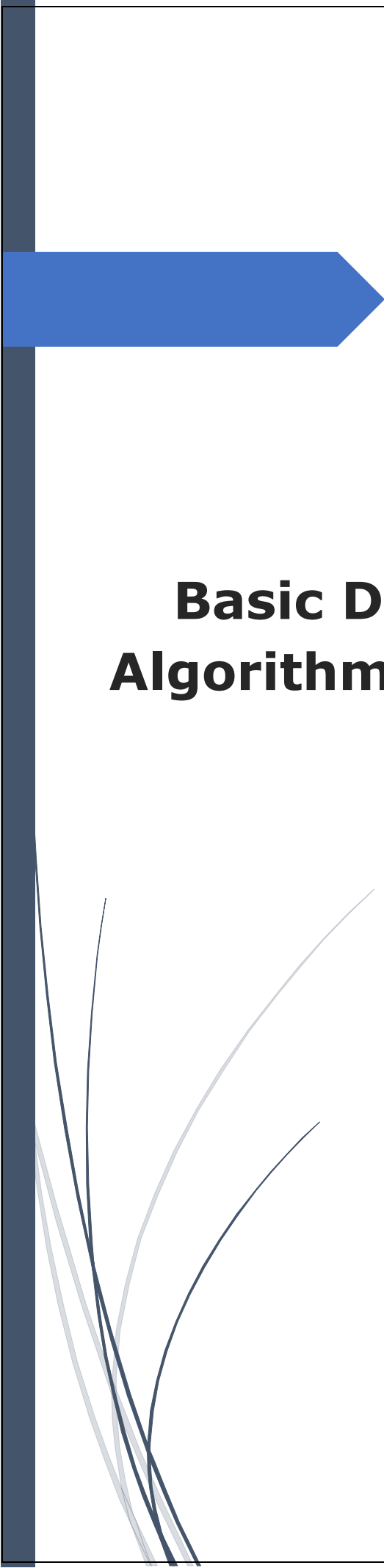




Basic Data Structures and Algorithm Notes for Beginners

Vimalathas Vithusan

Table of Contents

Abstract Data Types (ADTs):	2
Arrays:	3
Time complexity:	5
Linear and Binary Search:	7
Bubble Sort:	9
Selection Sort:	11
Insertion sort:	13
Quick sort:	15
Merge Sort:	17
Linked List:	20
Stack:	22
Queue:	24
Tree:	26

Abstract Data Types (ADTs):

1. **Definition:**

Abstract Data Types (ADTs) are high-level specifications of a set of operations that can be performed on a data structure.

2. **Encapsulation:**

ADTs encapsulate data, hiding the internal details of how the data is stored and processed.

3. **Operations:**

ADTs define a set of operations that can be performed on the encapsulated data. Common operations include insertion, deletion, retrieval, and traversal.

4. **Implementation Independence:**

ADTs are designed to be independent of specific implementation details. Different data structures and algorithms can be used to implement the same ADT.

5. **Mathematical Abstraction:**

ADTs are often described using mathematical abstractions, focusing on the behavior rather than the implementation.

6. **Examples:**

Examples of ADTs include Stack, Queue, List, Map (Dictionary), and Set.

7. **Behavior Specification:**

ADTs specify the behavior of operations rather than the specific steps of how those operations are carried out.

8. **Blueprint for Design:**

ADTs serve as blueprints for designing and understanding data structures. They guide the creation of concrete implementations.

9. **Object-Oriented Connection:**

ADTs are closely related to Object-Oriented Programming (OOP) principles, where classes and interfaces are used to define and implement abstract types.

10. **Flexibility:**

The flexibility of ADTs allows developers to choose appropriate implementations based on the requirements of a specific application.

11. **Formal Specification:**

Mathematical concepts, such as set theory, may be used to formally specify ADTs, enhancing precision and clarity.

Arrays:

1. **Definition:**

An array is a collection of elements, each identified by an index or a key. It is a linear data structure.

2. **Indexing:**

Elements in an array are stored at contiguous memory locations, and each element can be accessed using its index. The index starts from 0.

3. **Homogeneous:**

Arrays store elements of the same data type, making them homogeneous.

4. Declaration and Initialization:

Declaration:

```
datatype[] arrayName;
```

Initialization:

```
datatype[] arrayName = new datatype[size];
```

```
datatype[] arrayName = {element1, element2, ...};
```

5. **Size:**

The size of an array is fixed during declaration and cannot be changed during runtime.

6. **Random Access:**

Arrays provide constant-time access to any element using its index, enabling efficient random access.

7. **Advantages:**

- Simplifies access to elements through indexing.
- Allows efficient storage and retrieval of elements.
- Well-suited for scenarios where the size is known and constant.

8. **Disadvantages:**

- Fixed size: Difficult to change the size once declared.
- Inefficient for insertions and deletions in the middle (shifting required).
- Memory management: Requires contiguous memory space.

9. **Common Operations:**

- Accessing an element: `array[index]`
- Updating an element: `array[index] = newValue`
- Finding the length: `array.length`
- Iterating through elements: Using loops like `for` or `foreach`
-

10. Multi-dimensional Arrays:

- Arrays can have multiple dimensions (2D, 3D, etc.).
- Declaration:
`datatype[][] arrayName = new datatype[rowSize][columnSize];``

11. Dynamic Arrays:

Dynamic arrays, like ArrayList in Java, allow dynamic resizing and offer more flexibility.

12. Use Cases:

- Storing and accessing elements with known indices.
- Implementing matrix operations.
- Efficiently managing collections of data when the size is fixed.

Time complexity:

1. Definition:

- Time Complexity:
Measures the amount of time an algorithm takes with respect to its input size.
- Big O Notation (O):
Describes the upper bound of an algorithm's running time.

2. Factors Influencing Time Complexity:

- Input Size: Number of elements in the input.
- Algorithm Steps: Count of basic operations.
- Hardware and Environment: Execution speed varies.

3. Types of Time Complexity:

- $O(1)$: Constant time (e.g., accessing an array element).
- $O(\log n)$: Logarithmic time (e.g., binary search).
- $O(n)$: Linear time (e.g., traversing an array).
- $O(n \log n)$: Common in efficient sorting algorithms.
- $O(n^2)$: Quadratic time (e.g., nested loops).
- $O(2^n)$: Exponential time (highly inefficient).

4. Best, Average, and Worst Case:

- Best Case: Minimum time for any input (rarely used).
- Average Case: Expected time for a random input.
- Worst Case: Maximum time for any input (often analyzed).

5. Time Complexity Analysis:

- Evaluate basic operations as a function of input size.
- Disregard constants and focus on the dominant term.
- Asymptotic analysis for large input sizes.

6. Common Time Complexities:

- $O(1)$: Constant time (ideal).
- $O(\log n)$: Logarithmic time (efficient).
- $O(n)$: Linear time (typical).
- $O(n \log n)$: Efficient sorting algorithms.
- $O(n^2)$: Quadratic time (careful with large inputs).

7. Asymptotic Analysis:

- Focus on growth rate for infinite input.
- Identify dominant term in complexity function.

8. Big O Notation Examples:

- $O(1)$: Constant time.
- $O(n)$: Linear time.
- $O(\log n)$: Logarithmic time.
- $O(n^2)$: Quadratic time.
- $O(n \log n)$: Log-linear time.

9. Choosing Efficient Algorithms:

- Prefer lower time complexities for large inputs.
- Consider trade-offs with space complexity.

10. Practical Considerations:

- Analyze and compare time complexities.
- Select algorithms based on application requirements.
- Understand theoretical efficiency for scaling.

Linear and Binary Search:

Linear Search

1. Definition:

- Simple search algorithm.
- Sequentially checks each element in a list.

2. Steps:

- Start from the beginning.
- Compare each element with the target.
- Stop when the element is found or reach the end.

3. Time Complexity:

- Worst Case: $O(n)$ - Linear time.
- Best Case: $O(1)$ - Constant time (if the element is the first).

4. Use Cases:

- Small datasets.
- Unordered lists.

5. Advantages:

- Simplicity and ease of implementation.
- Works on unsorted lists.

6. Disadvantages:

- Inefficient for large datasets.
- May traverse the entire list.

Binary Search

1. Definition:

- Efficient search algorithm for sorted lists.
- Divides the search range by half in each step.

2. Steps:

- Compare the target with the middle element.
- Narrow the search range based on the comparison.
- Repeat until the element is found or the range is empty.

3. Time Complexity:

- Worst Case: $O(\log n)$ - Logarithmic time.
- Best Case: $O(1)$ - Constant time (if the element is in the middle).

4. Use Cases:

- Large sorted datasets.
- Maintained sorted lists.

5. Advantages:

- Efficient for large datasets.
- Reduces the search space quickly.

6. Disadvantages:

- Requires a sorted list.
- More complex to implement than linear search.

Note: Binary search is significantly more efficient than linear search for large sorted datasets, making it the preferred choice when applicable.

Bubble Sort:

1. Algorithm:

- Bubble Sort works by repeatedly stepping through the list of elements.
- It compares adjacent elements and swaps them if they are in the wrong order.
- The pass through the list is repeated until the list is sorted.

2. Passes:

- Each pass of the algorithm compares adjacent elements in the list and may perform multiple swaps.

3. Complexity:

- Bubble Sort has a time complexity of $O(n^2)$, where n is the number of elements in the list.
- In the worst case, when the list is in reverse order, Bubble Sort will make $n(n-1)/2$ comparisons.

4. Stability:

- Bubble Sort is a stable sorting algorithm. Stable sorting algorithms maintain the relative order of equal elements.

5. Adaptivity:

- It is not an adaptive sorting algorithm. Adaptive sorting algorithms perform better when the input is partially sorted.

6. Performance:

- Bubble Sort is not efficient for large lists and is mainly used for educational purposes or when the list is almost sorted.

7. Implementation:

- The algorithm involves iterating through the list multiple times, comparing elements, and swapping them if necessary.

8. Optimization:

- The algorithm can be optimized by stopping the algorithm if the inner loop doesn't perform any swaps in a pass, indicating that the list is already sorted.

Here's a simple example of Bubble Sort in Java:

```
public class BubbleSort {  
    public static void main(String[] args) {  
        int[] array = {64, 34, 25, 12, 22, 11, 90};  
  
        bubbleSort(array);  
  
        System.out.println("Sorted array:");  
  
        for (int value : array)  
            System.out.print(value + " ");  
    }  
  
    static void bubbleSort(int[] array) {  
        int n = array.length;  
        for (int i = 0; i < n-1; i++) {  
            for (int j = 0; j < n-i-1; j++) {  
                if (array[j] > array[j+1]) {  
                    // swap temp and arr[i]  
                    int temp = array[j];  
                    array[j] = array[j+1];  
                    array[j+1] = temp;  
                }  
            }  
        }  
    }  
}
```

Selection Sort:

1. Algorithm:

- Selection Sort divides the array into two parts: sorted and unsorted.
- It repeatedly selects the minimum element from the unsorted part and swaps it with the first element of the unsorted part.
- The sorted part grows, and the unsorted part shrinks until the entire array is sorted.

2. Selection Process:

- During each iteration, the algorithm finds the minimum element in the unsorted part.
- It then swaps this minimum element with the first element in the unsorted part.

3. Complexity:

- Selection Sort has a time complexity of $O(n^2)$, where n is the number of elements in the array.
- It is generally inefficient for large lists and is not suitable for large datasets.

4. Stability:

- Selection Sort is not stable. Stability in sorting means that the relative order of equal elements is maintained.

5. In-Place Sorting:

- Selection Sort is an in-place sorting algorithm, meaning it doesn't require additional memory space for sorting.

6. Comparison with Bubble Sort:

- Selection Sort and Bubble Sort have similar time complexities, but Selection Sort generally performs fewer swaps.

7. Implementation:

- The algorithm consists of nested loops, with the outer loop selecting the minimum element in each iteration and the inner loop iterating over the unsorted part.

Here's a simple example of Selection Sort in Java:

```
public class SelectionSort {
    public static void main(String[] args) {
        int[] array = {64, 25, 12, 22, 11};

        selectionSort(array);

        System.out.println("Sorted array:");
        for (int value : array) {
            System.out.print(value + " ");
        }
    }

    static void selectionSort(int[] array) {
        int n = array.length;

        for (int i = 0; i < n-1; i++) {
            int minIndex = i;

            // Find the minimum element in the unsorted part
            for (int j = i+1; j < n; j++) {
                if (array[j] < array[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap the found minimum element with the first element
            int temp = array[minIndex];
            array[minIndex] = array[i];
            array[i] = temp;
        }
    }
}
```

Insertion sort:

1. Algorithm:

- Insertion Sort works by dividing the array into two parts: sorted and unsorted.
- It iterates through the unsorted part and, in each iteration, selects an element and inserts it into its correct position in the sorted part.
- The process is repeated until the entire array is sorted.

2. Insertion Process:

- During each iteration, the algorithm selects an element from the unsorted part.
- It compares this element with the elements in the sorted part and inserts it into its correct position by shifting other elements to make space.

3. Complexity:

- Insertion Sort has a time complexity of $O(n^2)$, where n is the number of elements in the array.
- It performs well for small datasets and is often used in practice for partially sorted arrays or small datasets.

4. Stability:

- Insertion Sort is stable. Stability in sorting means that the relative order of equal elements is maintained.

5. In-Place Sorting:

- Insertion Sort is an in-place sorting algorithm, meaning it doesn't require additional memory space for sorting.

6. Adaptivity:

- It is an adaptive algorithm, meaning its performance is better when the input data is partially sorted.

7. Comparison with Selection Sort and Bubble Sort:

- Insertion Sort tends to perform better than Selection Sort and Bubble Sort, especially for small datasets.
- It is more efficient when the array is partially sorted.

8. Implementation:

- The algorithm uses nested loops, with the outer loop iterating over the unsorted part and the inner loop comparing and inserting elements.

Here's a simple example of Insertion Sort in Java:

```
public class InsertionSort {
    public static void main(String[] args) {
        int[] array = {64, 25, 12, 22, 11};

        insertionSort(array);

        System.out.println("Sorted array:");
        for (int value : array) {
            System.out.print(value + " ");
        }
    }

    static void insertionSort(int[] array) {
        int n = array.length;

        for (int i = 1; i < n; ++i) {
            int key = array[i];
            int j = i - 1;

            // Move elements greater than key to one position ahead
            while (j >= 0 && array[j] > key) {
                array[j + 1] = array[j];
                j = j - 1;
            }

            // Insert the key into its correct position
            array[j + 1] = key;
        }
    }
}
```

Quick sort:

1. Algorithm:

- QuickSort divides the array into two parts: elements less than a chosen pivot and elements greater than the pivot.
- It then recursively applies the same process to the sub-arrays until the entire array is sorted.

2. Partitioning:

- The partitioning step involves selecting a pivot element and rearranging the array so that elements less than the pivot are on its left, and elements greater than the pivot are on its right.

3. Pivot Selection:

- The choice of the pivot greatly influences the performance of QuickSort. Common strategies include selecting the first element, the last element, the middle element, or a random element as the pivot.

4. Complexity:

- On average, QuickSort has a time complexity of $O(n \log n)$.
- In the worst case (rare with proper pivot selection), it can degrade to $O(n^2)$, but this is uncommon in practice.

5. In-Place Sorting:

- QuickSort is an in-place sorting algorithm, meaning it doesn't require additional memory space for sorting.

6. Stability:

- QuickSort is not stable. Stability in sorting means that the relative order of equal elements is maintained.

7. Adaptivity:

- QuickSort is not adaptive. Adaptive sorting algorithms perform better when the input is partially sorted.

8. Parallelization:

- QuickSort can be easily parallelized, making it suitable for parallel processing.

9. Real-World Usage:

- QuickSort is widely used in practice and is often the default sorting algorithm in programming libraries and languages.

10. Comparison with Other Sorting Algorithms:

- QuickSort is generally more efficient than Bubble Sort, Insertion Sort, and Selection Sort, especially for large datasets.

Here's a simple example of QuickSort in Java:

```
public class QuickSort {
    public static void main(String[] args) {
        int[] array = {64, 25, 12, 22, 11};

        quickSort(array, 0, array.length - 1);

        System.out.println("Sorted array:");
        for (int value : array) {
            System.out.print(value + " ");
        }
    }

    static void quickSort(int[] array, int low, int high) {
        if (low < high) {
            // Partition the array, and get the pivot index
            int pivotIndex = partition(array, low, high);

            // Recursively sort the sub-arrays
            quickSort(array, low, pivotIndex - 1);
            quickSort(array, pivotIndex + 1, high);
        }
    }

    static int partition(int[] array, int low, int high) {
        int pivot = array[high];
        int i = (low - 1);

        for (int j = low; j < high; j++) {
            if (array[j] < pivot) {
                i++;

                // Swap array[i] and array[j]
                int temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
        }

        // Swap array[i+1] and array[high] (pivot)
        int temp = array[i + 1];
        array[i + 1] = array[high];
        array[high] = temp;

        return i + 1;
    }
}
```

Merge Sort:

1. Algorithm:

- Merge Sort divides the array into two halves until each sub-array contains only one element.
- It then recursively merges these sub-arrays, sorting them in the process.

2. Divide and Conquer:

- The divide-and-conquer approach involves breaking the problem into smaller sub-problems, solving them, and then combining the solutions.

3. Merge Operation:

- The merge step combines two sorted sub-arrays into a single sorted array.
- It compares elements from the two sub-arrays and merges them in sorted order.

4. Complexity:

- Merge Sort has a consistent time complexity of $O(n \log n)$ for all cases (best, average, and worst).
- It performs well for large datasets and is more efficient than many other sorting algorithms.

5. Stability:

- Merge Sort is stable. Stability in sorting means that the relative order of equal elements is maintained.

6. Adaptivity:

- Merge Sort is not adaptive. Adaptive sorting algorithms perform better when the input is partially sorted.

7. In-Place vs. Out-of-Place:

- Merge Sort can be implemented as an in-place or out-of-place algorithm.
- The standard recursive implementation is out-of-place and requires additional memory for merging.
- A bottom-up iterative approach can be used for in-place merging.

8. Parallelization:

- Merge Sort can be easily parallelized, making it suitable for parallel processing.

9. Real-World Usage:

- Merge Sort is widely used in practice and is often used as the default sorting algorithm in programming libraries and languages.

Here's a simple example of Merge Sort in Java:

```
public class MergeSort {
    public static void main(String[] args) {
        int[] array = {64, 25, 12, 22, 11};

        mergeSort(array, 0, array.length - 1);

        System.out.println("Sorted array:");
        for (int value : array) {
            System.out.print(value + " ");
        }
    }

    static void mergeSort(int[] array, int left, int right) {
        if (left < right) {
            // Find the middle point
            int mid = (left + right) / 2;

            // Recursively sort both halves
            mergeSort(array, left, mid);
            mergeSort(array, mid + 1, right);

            // Merge the sorted halves
            merge(array, left, mid, right);
        }
    }

    static void merge(int[] array, int left, int mid, int right) {
        // Calculate sizes of two sub-arrays to be merged
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Create temporary arrays
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];

        // Copy data to temporary arrays
        for (int i = 0; i < n1; ++i) {
            leftArray[i] = array[left + i];
        }
        for (int j = 0; j < n2; ++j) {
            rightArray[j] = array[mid + 1 + j];
        }

        // Merge the temporary arrays
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
```

```
    if (leftArray[i] <= rightArray[j]) {  
        array[k] = leftArray[i];  
        i++;  
    } else {  
        array[k] = rightArray[j];  
        j++;  
    }  
    k++;  
}  
  
// Copy remaining elements of leftArray, if any  
while (i < n1) {  
    array[k] = leftArray[i];  
    i++;  
    k++;  
}  
  
// Copy remaining elements of rightArray, if any  
while (j < n2) {  
    array[k] = rightArray[j];  
    j++;  
    k++;  
}  
}
```

Linked List:

1. Linked List Basics:

- A linked list is a linear data structure where elements are stored in nodes.
- Each node contains data and a reference (or link) to the next node in the sequence.

2. Types of Linked Lists:

- Singly Linked List: Each node points to the next node.
- Doubly Linked List: Each node points to both the next and the previous nodes.
- Circular Linked List: The last node points back to the first node, forming a loop.

3. Advantages:

- Dynamic Size: Linked lists can easily grow or shrink in size.
- Efficient Insertion and Deletion: Inserting or deleting elements in a linked list can be more efficient than arrays.

4. Disadvantages:

- Sequential Access: Random access is not as efficient as in arrays.
- Extra Memory: Requires extra memory for storing references.

5. Common Operations:

- Insertion:
Adding a new node to the list.
- Deletion:
Removing a node from the list.
- Traversal:
Iterating through the list to access or modify elements.

6. Traversal Methods:

- Iterative Traversal: Using loops to navigate through the list.
- Recursive Traversal: Using recursive functions to traverse the list.

7. Dummy Nodes:

- A dummy node (sentinel node) is a special node added to the list for various purposes, such as simplifying insertion or handling edge cases.

8. Cycle Detection:

- Checking whether a linked list contains a cycle (loop) to avoid infinite loops during traversal.

9. Applications:

- Linked lists are used in various applications, including implementation of stacks, queues, symbol tables, and memory allocation.

10. **Complexity:**

- The time complexity for common operations on a linked list depends on the type of operation and the type of linked list.
- Singly and doubly linked lists have different performance characteristics.

Here's a simple example of a singly linked list in Java:

```
class Node {
    int data;
    Node next;

    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

public class LinkedList {
    Node head;

    public void insert(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    public void display() {
        Node current = head;
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        LinkedList list = new LinkedList();
        list.insert(3);
        list.insert(7);
        list.insert(12);

        System.out.print("Linked List: ");
        list.display();
    }
}
```

Stack:

1. Definition:

- A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, where the last element added is the first one to be removed.

2. Basic Operations:

- Push: Adding an element to the top of the stack.
- Pop: Removing the element from the top of the stack.
- Peek (or Top): Retrieving the element at the top without removal.
- isEmpty: Checking if the stack is empty.

3. Key Characteristics:

- Elements are added and removed from the same end (top).
- Access to elements is restricted to the top of the stack.

4. Real-world Analogy:

- Analogous to a stack of plates where you add a plate to the top and remove the topmost plate.

5. Common Implementations:

- Implemented using arrays or linked lists.
- Array implementation requires specifying a maximum size.

6. Applications:

- Memory management (function call stack).
- Undo mechanisms in software applications.
- Parsing expressions in compilers.

7. Complexity:

- Time complexity for push and pop operations is $O(1)$.

8. Example (Java):

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // Pushing elements onto the stack
        stack.push(10);
        stack.push(20);
        stack.push(30);

        // Popping elements from the stack
        System.out.println("Pop: " + stack.pop());

        // Peeking at the top element
        System.out.println("Top: " + stack.peek());

        // Checking if the stack is empty
        System.out.println("Is Empty: " + stack.isEmpty());
    }
}
```

Queue:

1. Definition:

- A queue is a linear data structure that follows the First In, First Out (FIFO) principle, where the first element added is the first one to be removed.

2. Basic Operations:

- Enqueue (or Insert): Adding an element to the back (rear) of the queue.
- Dequeue (or Remove): Removing the element from the front (front) of the queue.
- Front: Retrieving the element at the front without removal.
- Rear: Retrieving the element at the back without removal.
- isEmpty: Checking if the queue is empty.

3. Key Characteristics:

- Elements are added at the rear and removed from the front.
- Access is restricted to the front and rear ends of the queue.

4. Real-world Analogy:

- Analogous to a line of people waiting for a service where the person at the front is served first.

5. Common Implementations:

- Implemented using arrays or linked lists.
- Circular arrays are often used to efficiently use space.

6. Types of Queues:

- Linear Queue: Elements are added at the rear and removed from the front.
- Circular Queue: Elements wrap around in a circular manner, utilizing the entire array.
- Priority Queue: Elements are assigned priorities, and the element with the highest priority is served first.

7. Applications:

- Task scheduling in operating systems.
- Print job management in printers.
- Breadth-first search in graph algorithms.

8. Complexity:

- Time complexity for enqueue and dequeue operations is $O(1)$.

9. Example (Java):

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();

        // Enqueuing elements to the rear
        queue.add(10);
        queue.add(20);
        queue.add(30);

        // Dequeuing elements from the front
        System.out.println("Dequeue: " + queue.poll());

        // Peeking at the front element
        System.out.println("Front: " + queue.peek());

        // Checking if the queue is empty
        System.out.println("Is Empty: " + queue.isEmpty());
    }
}
```

Tree:

1. Definition:

- A tree is a hierarchical data structure that consists of nodes connected by edges. Each node has a value and zero or more child nodes, except for the root, which has no parent.

2. Basic Components:

- Node: Fundamental building block containing data.
- Edge: Connection between nodes.
- Root: Topmost node, serving as the starting point.
- Parent: A node with child nodes.
- Child: Nodes connected to a parent.
- Leaf: Nodes with no children.

3. Types of Trees:

- Binary Tree: Each node has at most two children.
- Binary Search Tree (BST): Binary tree with the property that the left child is less than the parent, and the right child is greater.
- Balanced Tree: Ensures that the height difference between the left and right subtrees is limited.
- N-ary Tree: Nodes can have more than two children.
- Trie: Specialized tree for storing dynamic sets or associative arrays.

4. Tree Traversal:

- In-Order: Left subtree, root, right subtree.
- Pre-Order: Root, left subtree, right subtree.
- Post-Order: Left subtree, right subtree, root.
- Level-Order: Nodes visited level by level, left to right.

5. Tree Operations:

- Insertion: Adding a new node to the tree.
- Deletion: Removing a node from the tree.
- Searching: Locating a specific value in the tree.

6. Properties:

- Height: The length of the longest path from the root to a leaf.
- Depth: The length of the path from a node to the root.
- Balanced Tree: Ensures relatively uniform tree structure, avoiding degeneration into a linked list.

7. Applications:

- File Systems: Representing hierarchical file structures.
- Database Indexing: Efficient searching and retrieval.
- Expression Evaluation: Parsing and evaluating mathematical expressions.

8. Complexity:

- Time Complexity: Depends on the operation and the tree's structure.
- Space Complexity: Generally $O(n)$, where n is the number of nodes.

9. Example (Java):

```
class TreeNode {
    int data;
    TreeNode left, right;

    public TreeNode(int data) {
        this.data = data;
        this.left = this.right = null;
    }
}

public class BinaryTree {
    TreeNode root;

    public BinaryTree(int rootData) {
        this.root = new TreeNode(rootData);
    }

    // Additional tree operations and traversals can be implemented here
}
```

Thank you!